

Advanced Natural Language Processing

Final Examination Preparation: Questions and Answers

This set covers RNNs, LSTMs/GRUs, encoder–decoder models, attention, Transformers, BERT, the GPT family, LLaMA, and the remaining topics of the LLM lecture (scaling laws, long context, Mixture of Experts, quantization, PEFT, pruning, distillation, instruction tuning, alignment, prompting, RAG, agents, and evaluation). Questions follow the conceptual patterns used in the course: advantages/disadvantages, what a method solves, what it cannot solve, what new problems it created, when to use and not use it, and scenario-based reasoning.

Contents

1 Recurrent Neural Networks	2
2 LSTM and GRU	3
3 Encoder–Decoder (Seq2Seq) Models and Attention	4
4 The Transformer	6
5 BERT	8
6 The GPT Family	10
7 LLaMA	11
8 Pretraining Objectives, Data, and Scaling Laws	13
9 Long Context and Mixture of Experts	14
10 Quantization, PEFT, Pruning, and Distillation	15
11 Instruction Tuning and Alignment	17
12 Prompting and In-Context Learning	18
13 Retrieval-Augmented Generation	19
14 Reasoning Models and Agentic AI	20
15 Evaluation of LLMs	21
16 Integrated Scenario Questions	22

1 Recurrent Neural Networks

Q1. What problem do RNNs solve that feedforward networks and CNNs cannot?

A. Feedforward and convolutional networks map a fixed-size input to a fixed-size output using a fixed number of computational steps, so they cannot naturally handle sequences of arbitrary length. An RNN applies the same parameters W , U , V at every time step and carries a hidden state forward, so it can consume or produce sequences of any length. This makes one-to-many, many-to-one, and many-to-many mappings possible with a single architecture. It also lets the model condition each prediction on all previously seen inputs rather than on a fixed window.

Q2. What are the main drawbacks of a simple (vanilla) RNN?

A. A simple RNN can only see the past, not the future, so it cannot use right-hand context when producing a representation at time t . It must forget the same amount of history at every time step, because the same transition matrix is applied unconditionally, even though the task may require remembering some tokens far longer than others. Maintaining long-term memory is therefore difficult in practice. Finally, its computation is inherently sequential, which prevents parallelisation across time.

Q3. What new problem did training RNNs by backpropagation through time (BPTT) give rise to?

A. BPTT unrolls the network over time and multiplies the per-step Jacobians together as the gradient is propagated backward. If those per-step derivatives are consistently smaller than one, the product shrinks geometrically and the gradient reaching early time steps approaches zero, which is the vanishing gradient problem. Gradients that are effectively zero carry no useful information for parameter updates, so the model never learns long-range dependencies. The mirror-image failure, when the factors are consistently larger than one, is exploding gradients, which destabilises training and is usually handled by gradient clipping.

Q4. What problem can an RNN *not* solve, no matter how well it is trained?

A. It cannot be parallelised across the time dimension, because h_t requires h_{t-1} by construction. This is an architectural limitation, not a training limitation, so more data or better optimisation cannot remove it. On GPUs, which are bandwidth-optimised and reward large parallel matrix operations, this sequential dependency wastes most of the available throughput. That single limitation is the strongest practical reason the field moved to Transformers.

Q5. When is a simple RNN still a reasonable choice?

A. When sequences are short, the dependencies are local, and the deployment budget is tiny — for example a small character-level model on an embedded device. It is also a reasonable choice for strictly online or streaming settings where tokens must be consumed one at a time with constant memory per step, since a Transformer would need to re-attend over a growing context. In teaching and diagnostic settings it is useful because the recurrence is transparent. Outside these cases a gated RNN or a Transformer is almost always better.

Q6. When should an RNN *not* be used?

A. It should not be used when long-range dependencies matter, since the vanishing gradient problem makes those dependencies hard to learn. It should not be used when training throughput matters, because sequential processing leaves GPU capacity idle. It is also a poor choice when large-scale pretraining is available, since pretrained Transformer encoders and decoders

dominate on nearly every benchmark. Finally, avoid it for tasks needing bidirectional context unless a bidirectional variant is explicitly used.

Q7. Scenario: you build a part-of-speech tagger with a unidirectional RNN and accuracy on ambiguous words is poor. What would change if you used a bidirectional RNN instead?

A. A bidirectional RNN runs one forward pass and one backward pass and concatenates the two hidden states, so the representation at each position sees both left and right context. Ambiguities such as noun/verb readings of “book” are frequently resolved by the following word, which the unidirectional model never sees. Accuracy on exactly those cases would improve substantially. The cost is that the whole sentence must be available before tagging, so the model is no longer usable for streaming input.

Q8. Scenario: a streaming speech recogniser must emit words as the user speaks. What would happen if you replaced its unidirectional RNN with a bidirectional one?

A. The backward pass cannot start until the utterance ends, so the model would be unable to emit any output until the speaker stops. Offline accuracy would improve, but the latency requirement of the application would be violated. The correct solution is to keep unidirectionality and instead widen context in a permitted way, for example with a small look-ahead buffer or chunked bidirectional processing. This is a case where an architecturally better model is the wrong engineering choice.

Q9. Scenario: for text classification you take the last hidden state of an RNN as the sentence representation and results are weak on long reviews. What is going wrong and what is the fix?

A. The last hidden state is a fixed-size summary that must compress the entire sequence, and because of the vanishing gradient the early part of a long review contributes very little to it. The representation is therefore dominated by the final sentences. Replacing the last state with mean or max pooling over all hidden states, or adding an attention layer that learns a weighted pooling, distributes the signal across the sequence. Switching to an LSTM or a pretrained Transformer encoder addresses the same problem more fundamentally.

2 LSTM and GRU

Q10. What problem does the LSTM solve, and by what mechanism?

A. It solves the inability of a simple RNN to control how much history is retained, and thereby mitigates the vanishing gradient problem. It introduces a separate cell state C_t together with forget, input, and output gates computed from h_{t-1} and x_t . Because the cell state is updated by an element-wise multiplication with the forget gate plus an additive term, gradients can flow backward along the cell path with much less attenuation than through repeated matrix multiplication. The gates let the network forget different amounts at different time steps, which is exactly what the vanilla RNN cannot do.

Q11. Give the ordered computation inside one LSTM cell.

A. First compute the forget gate f_t , input gate i_t , and output gate o_t from h_{t-1} and x_t using sigmoid activations. Second, compute the candidate cell update $\tilde{C}_t$, usually with a tanh. Third, update the cell state as $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$. Fourth, produce the hidden state $h_t = o_t \odot \tanh(C_t)$.

Q12. What are the disadvantages of LSTMs?

A. They are still sequential, so they do not fix the parallelisation problem at all. They have roughly four times the parameters and computation of a simple RNN per time step, which makes them slower and more memory hungry. They mitigate but do not eliminate vanishing gradients, so very long dependencies (thousands of tokens) remain hard. They also add several gates and hyperparameters that make the model harder to reason about and tune.

Q13. What problems can an LSTM *not* solve?

A. It cannot provide constant-path access to arbitrary earlier tokens: information must still travel through every intermediate step, so the effective memory is finite. It cannot be parallelised over the sequence during training. It cannot give bidirectional context unless explicitly run in both directions. And it does not scale into the regime where large pretrained models operate, which is why the field moved to attention-based architectures.

Q14. How does a GRU differ from an LSTM, and when would you prefer it?

A. A GRU merges the cell state and hidden state and uses two gates (update and reset) instead of three, so it has fewer parameters and is faster to train. Empirically it performs comparably to an LSTM on many sequence tagging and classification tasks. Prefer it when data is limited, when compute or latency is constrained, or as a strong default baseline; the course notes a bidirectional GRU as a good starting point for many sequence tagging tasks. Prefer an LSTM when the task requires very long memory and you have enough data to fit the extra parameters.

Q15. Scenario: you remove the forget gate from an LSTM and hard-code it to 1. What would happen?

A. The cell state would become a pure running accumulator, $C_t = C_{t-1} + i_t \odot \tilde{C}_t$, with no mechanism to discard stale information. On short sequences it may still train, but on long sequences the cell state magnitudes grow without bound and saturate the tanh in the output path, degrading the hidden state. The model would also lose the ability to reset context at sentence or document boundaries. In effect you would recover much of the “forget the same amount at each step” weakness the LSTM was designed to remove.

Q16. Scenario: your LSTM language model trains stably but its loss suddenly spikes to NaN. What is the likely cause and remedy?

A. This is the exploding gradient counterpart of the vanishing gradient problem: repeated multiplication of Jacobians with factors greater than one produces very large updates. A single bad batch can then push weights far outside their useful range. The standard remedy is gradient clipping by global norm, together with a lower learning rate and careful initialisation. Note that gating alone does not prevent explosion — it addresses vanishing, not exploding, gradients.

3 Encoder–Decoder (Seq2Seq) Models and Attention

Q17. What problem does the encoder–decoder architecture solve?

A. It solves the mismatch between input and output lengths in tasks such as machine translation, where the source and target sentences have different lengths and different word orders. The encoder RNN compresses the source into a hidden representation, and the decoder RNN generates the target autoregressively from it. This decouples reading from writing, so the model need not emit one output token per input token. It made end-to-end neural machine translation feasible.

Q18. What problem did the basic RNN encoder–decoder give rise to?

A. It creates an information bottleneck: the entire source sentence must be squeezed into a single fixed-size context vector, the encoder’s last hidden state. As sentences grow longer, this vector cannot preserve enough detail and translation quality degrades sharply with length. Early source words are also the most attenuated, since they must survive the largest number of recurrent steps. Attention was introduced precisely to remove this bottleneck.

Q19. What problem does attention over encoder states solve, and how?

A. It removes the fixed-size bottleneck by letting the decoder look at *all* encoder hidden states rather than only the last one. At each decoding step the decoder computes a similarity score between its current state and every encoder state, normalises those scores with a softmax, and forms a weighted sum of encoder states as the context vector. Because the weights change per output token, the decoder can focus on the source positions relevant right now, which handles reordering and long sentences. It also provides a soft alignment that can be inspected.

Q20. What can attention over an RNN encoder *not* fix?

A. It does not remove the sequential nature of the RNN, so training still cannot be parallelised across time. Gradients still travel through the recurrence for the encoder’s own representation building. It also introduces a cost proportional to (source length $\times$ target length) for the alignment computation. So attention solved the bottleneck but left the throughput problem, which is what motivated “Attention Is All You Need”.

Q21. What is teacher forcing, and what problem does it create?

A. During training the decoder is fed the ground-truth previous token rather than its own prediction, which stabilises and accelerates learning because errors do not compound within a training sequence. At inference the model must instead feed back its own outputs autoregressively. This mismatch is exposure bias: the model has never been trained on the imperfect prefixes it will actually condition on. The result is error compounding in long generations, which teacher forcing alone cannot fix.

Q22. Scenario: an RNN encoder–decoder without attention translates short sentences well but collapses on 60-word legal sentences. What would have happened if attention had been included from the start?

A. The degradation with length would have been far smaller, because the decoder would draw a fresh context vector from all encoder states at every output step instead of relying on one compressed summary. Long-distance reordering, common in legal text, would be handled better since the alignment weights can jump to distant source positions. Training would cost slightly more per step due to the alignment computation. Quality on short sentences would be roughly unchanged, since there the bottleneck was never binding.

Q23. When should you choose an encoder–decoder model over a decoder-only model?

A. Choose encoder–decoder when there is a clearly separate source that should be read bidirectionally and a separate target to generate — translation, summarisation, or grammatical error correction. The encoder can attend in both directions over the source, which a causal decoder cannot do. Choose decoder-only when the task is open-ended continuation, when you want one model to handle many tasks through prompting, or when you want the simplicity and scaling behaviour that made GPT-style models dominant. In practice large decoder-only models now match encoder–decoder models on many seq2seq tasks by putting the source in the prompt.

4 The Transformer

Q24. What two problems does the Transformer solve simultaneously?

A. It removes recurrence, so all positions in a sequence are processed in parallel and training makes full use of GPU throughput. It also gives every token a direct, constant-length path to every other token through self-attention, so long-range dependencies no longer have to survive many recurrent steps. Together these fix the two central weaknesses of RNN-based seq2seq: slow training and weak long-distance modelling. This is what made large-scale pretraining practical.

Q25. Explain the Query, Key, Value formulation and why it is called *self*-attention.

A. Each input embedding passes through three learned linear projections producing a query, a key, and a value; the analogy is information retrieval, where the query is the search string, keys are the indexed titles, and values are the returned items. The relevance of token j to token i is the scaled dot product of q_i and k_j , and the output for i is the softmax-weighted sum of the values. It is called self-attention because queries, keys, and values all come from the *same* sequence, so the layer models relations among the words of one sentence rather than between a separate query and a separate document. This is what lets the model resolve, for example, whether “it” refers to “the animal” or “the street”.

Q26. Why are the attention scores divided by $\sqrt{d_k}$, and what would happen without it?

A. For queries and keys with roughly unit-variance components, the dot product of two d_k -dimensional vectors has variance on the order of d_k , so scores grow with dimension. Without the $1/\sqrt{d_k}$ scaling, large-magnitude scores push the softmax into a near one-hot regime where its gradient is almost zero. Training would then stall or become unstable, especially for large d_k . Scaling keeps the score distribution in a range where the softmax has useful gradients.

Q27. Why use multi-head attention instead of one large attention head?

A. Splitting the model dimension into several heads, each of dimension smaller than the embedding (64 in the original paper with $d_{\text{model}} = 512$), lets different heads attend to different kinds of relations — syntactic dependency, coreference, positional proximity — in different learned subspaces. A single head produces one averaged attention distribution and must trade these off against each other. The total computation is comparable because the per-head dimension shrinks. The heads’ outputs are concatenated and passed through an output projection.

Q28. Why does the Transformer need positional encoding, and what would happen if it were removed?

A. Self-attention is permutation-equivariant: it computes a weighted sum over a set, so “dog bites man” and “man bites dog” would produce identical token representations. Positional encodings inject order by adding a position-dependent vector of the same dimension as the embedding. Without them the model would become a bag-of-words model and would fail on any task where word order carries meaning. Performance on translation, syntax, and negation would collapse while topic-level tasks might partly survive.

Q29. Why did the original paper use sinusoidal positional encodings, and what are the trade-offs against learned ones?

A. A sinusoid of the form $\sin(k/n^{2i/d})$ and $\cos(k/n^{2i/d})$ with $n = 10000$ is periodic and defined for every position, so the encoding of position k is the same regardless of sequence length and

positions beyond the training length still receive a value. Relative offsets can also be expressed as linear functions of the encodings. Learned absolute embeddings are also possible and perform comparably in practice, but they are undefined beyond the maximum trained position, so they cannot extrapolate. The sinusoidal choice therefore buys length generalisation at no measured cost in quality.

Q30. What is the purpose of the residual connections and the “Add & Norm” layers?

A. The residual adds the sublayer’s input back to its output, which preserves the original information and gives gradients a short path to early layers, mitigating vanishing gradients in deep stacks. The normalisation step then rescales the summed activations across the feature axis, which stabilises and speeds up training. Without residuals, deep Transformer stacks are very difficult to optimise. Every attention and feed-forward sublayer is wrapped this way.

Q31. Why does the Transformer use layer normalisation rather than batch normalisation?

A. Batch normalisation computes mean and variance of each neuron across the mini-batch, so its statistics depend on batch composition and on which positions are padding. With variable-length sequences the per-position batch statistics are unreliable, and at inference with batch size one they are unusable without stored running estimates. Layer normalisation instead normalises across the features of a single token, so it is independent of batch size and of sequence length. Substituting batch normalisation would make training noisy and would break the train/inference consistency that NLP workloads require.

Q32. What does masked multi-head attention in the decoder do, and what happens if the mask is removed?

A. The mask sets attention scores to positions after the current one to $-\infty$ before the softmax, so each position can attend only to itself and earlier positions. This preserves the autoregressive property and matches training to inference, where future tokens do not exist. Removing the mask lets the model see the token it is supposed to predict, so training loss falls to near zero while generation at inference time is nonsense. It is a classic label-leakage failure.

Q33. What is encoder–decoder (cross) attention, and how does it differ from self-attention?

A. In cross-attention the keys and values come from the encoder output while the queries come from the decoder’s current representation. This lets each generated token look back at the source sequence, playing the same role attention played in RNN seq2seq. It differs from self-attention, where all three projections come from the same sequence. It is also not masked, since the entire source is legitimately available.

Q34. What problem does Byte-Pair Encoding solve, and what does it not solve?

A. Word-level vocabularies are enormous and still produce out-of-vocabulary tokens for inflections and compounds (German *Krankenhaus*, English *low/lower/lowest*). BPE builds a vocabulary by repeatedly merging the most frequent adjacent symbol pairs, so frequent words stay whole while rare words decompose into subwords, giving a fixed, modest vocabulary with no OOV tokens. What it does not solve is meaningful morphological segmentation — merges are frequency-driven, not linguistic — and it can fragment low-resource languages, numbers, and code badly, inflating sequence length and cost. It also makes character-level tasks such as counting letters awkward.

Q35. What new problems did the Transformer itself give rise to?

A. Self-attention costs time and memory quadratic in sequence length, which caps context and motivated later work such as FlashAttention and sparse attention. Removing recurrence removed a useful inductive bias about locality and order, so Transformers need far more data to reach comparable performance on small datasets. Model and data scale exploded, making training expensive, energy-intensive, and concentrated in a few organisations. Positional generalisation beyond the trained length also became a research problem in its own right.

Q36. When should a Transformer *not* be used?

A. Avoid it when the training set is small and no suitable pretrained checkpoint exists, since it will overfit where a GRU or a linear model would not. Avoid it for very long raw sequences under tight memory budgets, because of the quadratic attention cost. Avoid it when strict low-latency, low-power inference is required and a smaller recurrent or convolutional model meets the accuracy target. Finally, if the task is genuinely local (for example simple regular-expression-like pattern matching), the extra capacity buys nothing.

Q37. Scenario: a colleague reports that their Transformer trains $20\times$ faster per epoch than their LSTM but performs worse on a 5,000-sentence dataset. Explain.

A. The speed-up is expected, because the Transformer parallelises across positions while the LSTM cannot. The accuracy result is also expected: the LSTM's recurrence encodes a strong prior about sequential locality, whereas the Transformer must learn order and locality from data through positional encodings and attention. With only 5,000 sentences there is not enough signal to learn that structure, so the Transformer overfits. The right fix is to fine-tune a pretrained Transformer rather than to train one from scratch.

5 BERT

Q38. What problem does BERT solve?

A. Prior pretrained language models were unidirectional, so a token's representation could condition on only left or only right context, which is a poor fit for understanding tasks such as question answering and inference. BERT pretrains deep *bidirectional* representations by jointly conditioning on left and right context in all layers. A pretrained BERT can then be fine-tuned with just one extra output layer to reach state-of-the-art results across many tasks. It thereby replaced task-specific architectures with a single pretrain-then-fine-tune recipe.

Q39. Why can BERT not simply be trained with standard left-to-right language modelling?

A. If every layer is bidirectional, each token can see itself through the other direction, so predicting the next token becomes trivial and the model learns nothing. Bidirectionality and next-token prediction are therefore incompatible in a deep stack. Masked language modelling resolves this by hiding a subset of tokens and asking the model to reconstruct them from the surrounding context on both sides. This is the key design decision that makes deep bidirectional pretraining possible.

Q40. Describe BERT's masking scheme and explain the 80/10/10 split.

A. BERT selects 15% of tokens at random; of those, 80% are replaced by [MASK], 10% by a random token, and 10% are left unchanged. The [MASK] token never appears during fine-tuning, so training exclusively on it would create a pretrain/fine-tune mismatch; leaving 10%

unchanged reduces that mismatch. The 10% random replacement forces the model to maintain a good contextual representation of *every* token, since it cannot assume an unmasked token is correct. Without these two components the encoder would learn to copy unmasked inputs rather than to understand them.

Q41. What was Next Sentence Prediction for, and what criticism does it attract?

A. NSP trains the model on pairs where 50% of the time sentence B truly follows A and 50% of the time B is a random sentence, so that the model learns inter-sentence relationships needed by question answering and natural language inference, which masked language modelling alone does not capture. The criticism is that distinguishing a random sentence from a genuine continuation is largely a topic-detection task and is too easy, so it teaches little about discourse coherence. Later work (notably RoBERTa) removed NSP and improved results. It illustrates that an auxiliary objective must be hard enough to be informative.

Q42. What can BERT *not* do?

A. It cannot generate fluent free-form text, because it is an encoder trained to fill in masked positions rather than to model $p(x_t \mid x_{<t})$ autoregressively. It cannot be used directly for open-ended tasks like summarisation, translation, or dialogue without bolting on a decoder. Its context length is fixed and short (512 tokens in the original), so full documents must be chunked. It also has no in-context learning ability: adapting it to a new task requires gradient updates.

Q43. What are the disadvantages of BERT?

A. It is large by the standards of its time — 110M parameters for base (12 layers, 12 heads, 768 hidden) and 340M for large (24 layers, 16 heads, 1024 hidden) — which makes fine-tuning and serving expensive. Only 15% of tokens contribute to the loss per example, so pretraining is sample-inefficient compared with causal language modelling, which supervises every position. The [MASK] token creates a residual pretrain/fine-tune discrepancy. And every downstream task needs its own fine-tuned copy of the weights.

Q44. When should BERT be used and when should GPT be used?

A. Use BERT when the task is to *understand* a fixed input — classification, named entity recognition, extractive QA, sentence similarity, retrieval embeddings — especially when you have labelled data and want a small, fast, deterministic model. Use GPT-style decoder models when the task is to *produce* text, when the task set is open or changes often, or when you want zero- or few-shot behaviour without fine-tuning. A useful rule: bidirectional encoders for discriminative tasks, causal decoders for generative ones. If a task needs both a rich source representation and generation, an encoder–decoder is the natural middle ground.

Q45. Scenario: you must classify 10 million support tickets per day on a fixed CPU budget. BERT-large is accurate but too slow. What would you do?

A. Move down the size ladder first: BERT-base, then a distilled student such as DistilBERT, which is about 40% smaller while retaining roughly 97% of accuracy. Then apply post-training int8 quantization, which suits CPU inference well because integer matrix multiplication is fast and the model footprint shrinks. If throughput is still insufficient, apply structured pruning of heads or layers followed by a short fine-tune to recover accuracy. Using a generative LLM here would be the wrong direction entirely, since the task is a fixed-label classification with abundant training data.

Q46. Scenario: instead of fine-tuning BERT you freeze it and use its hidden states as features. What do you gain and lose?

A. You gain cheap training, since only a small classifier is learned, and you can precompute and cache embeddings once and reuse them across many tasks. You also avoid catastrophic forgetting and reduce overfitting risk on small datasets. You lose accuracy relative to full fine-tuning, because the representations are not adapted to the target task or domain. Feature extraction is therefore preferred when compute is tight, data is small, or many tasks share one encoder; fine-tuning is preferred when maximum accuracy on one task matters.

6 The GPT Family

Q47. What is the core design of GPT-1 and what problem did it address?

A. GPT-1 is a 12-layer decoder-only Transformer stack with masked multi-head self-attention, pretrained with causal (next-token) language modelling on unlabelled text and then fine-tuned per task. It addressed the scarcity of labelled data by showing that generative pretraining transfers broadly. Different tasks were handled by serialising the inputs into a single token sequence with delimiter tokens — premise/hypothesis for entailment, context/answer for multiple choice — so one architecture could serve all of them. This established the “pretrain generatively, adapt minimally” template.

Q48. What did GPT-2 change conceptually, not just in scale?

A. GPT-2 was a direct ten-fold scale-up of GPT-1 (1.5B versus about 120M parameters) with a correspondingly larger dataset, but its conceptual claim was that fine-tuning could be dropped altogether. The argument is that a sufficiently diverse Internet corpus already contains demonstrations of many tasks, so a sufficiently strong language model can perform them through the language-modelling objective alone, as reflected in the paper title “Language Models are Unsupervised Multitask Learners”. Its training set, WebText, was built by scraping only pages linked from Reddit posts with at least 3 karma before December 2017, roughly 45 million links and 40 GB of text. That karma filter is a cheap human quality signal, which is the practical lesson.

Q49. What did GPT-3 demonstrate, and at what cost?

A. GPT-3 showed that scaling to 175B parameters turns the model into a strong *in-context learner*: it can perform new tasks from a few examples in the prompt with no weight updates at all. It was trained on roughly 570 GB of filtered text amounting to about 300 billion tokens from CommonCrawl, WebText, Wikipedia, and two book corpora. The cost was extreme — on the order of hundreds of GPU-years and millions of dollars — and it was released only through an API rather than as open weights. GPT-3.5 and InstructGPT then converted this raw capability into a usable assistant, which is what triggered the current LLM boom.

Q50. What new problems did the GPT scaling trajectory give rise to?

A. Training became so expensive that it concentrated capability in a handful of well-funded labs, and weights stopped being published — GPT-2 was the last publicly available model in the line. Raw scaled language models are not aligned: they are fluent but not helpful, honest, or harmless, which created the whole instruction-tuning and RLHF agenda. Massive web training created data contamination, privacy, copyright, and bias problems. Finally, evaluating such general models became difficult because benchmarks may already appear in the training data.

Q51. What did GPT-4 add beyond scale?

A. It moved into multimodality, accepting image as well as text input, and it leaned heavily on instruction tuning with reinforcement learning from human and AI feedback. It substantially extended the context window (up to 32K tokens in the initial variants) and improved chat and reasoning behaviour. The emphasis therefore shifted from “bigger pretraining” to “better post-training and longer context”. This reflects the general trend that alignment and context, not raw parameter count alone, drive perceived quality.

Q52. What is the intrinsic disadvantage of causal (decoder-only) language modelling?

A. Each token is represented using only its left context, so the representation of an early token can never incorporate the disambiguating information that appears later. For pure understanding tasks this is a real handicap relative to a bidirectional encoder. Generation is also strictly sequential at inference, one token per forward pass, which limits latency. And because the model always predicts a plausible next token, it will confidently produce fluent but false continuations — the mechanistic root of hallucination.

Q53. Scenario: you need embeddings for a semantic search index. A team member proposes taking the last hidden state of a GPT model. What is the problem?

A. A causal model’s final-position state is optimised to predict the next token, not to summarise the whole input, and every token’s representation lacks right context. The resulting vectors tend to be dominated by the last few tokens and are poorly calibrated for cosine similarity. A bidirectional encoder trained or fine-tuned for sentence similarity (BERT-style, with mean pooling or a contrastive objective) is the appropriate tool. If a decoder model must be used, it should first be adapted with a contrastive embedding objective rather than used raw.

Q54. Scenario: what would have happened if GPT-2 had been trained on raw, unfiltered CommonCrawl instead of WebText?

A. Raw CommonCrawl contains large amounts of boilerplate, spam, machine-generated text, and duplicated pages, so a large fraction of the compute budget would be spent modelling low-value text. Sample quality and coherence would drop, and the model would more readily reproduce toxic and templated content. The WebText karma filter acted as a crowd-sourced quality proxy, which is the concrete instantiation of the course’s point that data quality beats data quantity. The counterfactual illustrates that at fixed compute, curation is a first-order design decision, not a preprocessing detail.

7 LLaMA

Q55. What problem does the LLaMA family address?

A. It addresses the closure and inefficiency of the GPT-3 generation: state-of-the-art models were API-only and were larger than their training data justified. LLaMA models are comparatively small but trained on far more tokens, following the compute-optimal insight, and their weights are made freely available. This made competitive LLMs reproducible and fine-tunable by academics and small companies. It is arguably the most widely used openly available LLM family, spanning three generations and several parameter sizes.

Q56. Which architectural changes does LLaMA make relative to the original Transformer decoder?

A. It replaces LayerNorm with RMSNorm and applies it *before* each sublayer (pre-normalisation) rather than after. It replaces absolute sinusoidal positional encodings with

Rotary Positional Embeddings applied to the queries and keys inside every attention layer. It replaces the ReLU feed-forward network with a SwiGLU gated feed-forward network. It also uses SentencePiece tokenisation, and the later 34B and 70B LLaMA 2 models use Grouped-Query Attention with a KV cache.

Q57. What does RMSNorm do differently from LayerNorm, and why?

A. LayerNorm subtracts the mean and divides by the standard deviation of a token's features, then applies a learned gain and bias. RMSNorm drops the mean-centring and the bias entirely, rescaling only by the root mean square of the features. This removes two reductions over the feature dimension per normalisation, which is a measurable saving at scale, and empirically the re-centring turns out to contribute little to stability. The trade-off is a slightly less expressive normalisation for a faster one.

Q58. Why does LLaMA apply normalisation before each sublayer instead of after?

A. With post-normalisation the residual stream is renormalised at every block, which shrinks the effective gradient path in very deep stacks and typically requires a careful learning-rate warm-up to avoid divergence. Pre-normalisation leaves a clean identity path through the residual stream, so gradients reach early layers largely undamped. This makes training deep models markedly more stable. The cost is that activation magnitudes in the residual stream can grow with depth, which a final normalisation before the output projection handles.

Q59. What problem do Rotary Positional Embeddings solve that absolute encodings do not?

A. Absolute encodings are added once to the input embedding, so positional information is diluted as it passes through layers, and the model sees absolute indices rather than distances. RoPE instead rotates the query and key vectors by an angle proportional to their position, so the dot product between them depends only on their *relative* offset. This injects position directly into every attention computation at every layer and makes the encoding of relative distance explicit. It also extrapolates to longer sequences far more gracefully than learned absolute embeddings, which are simply undefined past the trained maximum length.

Q60. What problem does Grouped-Query Attention solve? (Note the slide attribution carefully.)

A. During autoregressive decoding, previously computed keys and values are cached, and reading that KV cache from memory — not the arithmetic — dominates the cost, because modern GPUs compute far faster than they can move memory. Multi-Query Attention shares a single key/value head across all query heads, shrinking the cache dramatically but hurting quality; Grouped-Query Attention shares one key/value head per *group* of query heads, recovering most of the quality at most of the memory saving. It therefore targets memory bandwidth, not FLOPs. Note that the lecture slide lists GQA under “LLaMA 1 new concepts”, but GQA was in fact introduced in LLaMA 2 and only in the 34B and 70B variants; LLaMA 1 used standard multi-head attention.

Q61. Why SwiGLU instead of ReLU in the feed-forward block?

A. SwiGLU is a gated linear unit in which one linear branch is passed through a Swish activation and multiplied element-wise with a second linear branch, giving the network a learned, input-dependent gate on each hidden unit. This is more expressive than a fixed ReLU threshold and empirically yields better perplexity at equal parameter count. The cost is a third weight matrix in the feed-forward block, so implementations shrink the hidden width (typically to $\frac{2}{3} \cdot 4d$) to

keep the parameter budget constant. The trade-off is more matrix multiplications for better quality per parameter.

Q62. Scenario: you serve a 70B model to many concurrent users with long conversations, and you run out of GPU memory even though the weights fit. What is happening and what would you change?

A. The KV cache, not the weights, is consuming the memory: its size grows with (batch size $\times$ sequence length $\times$ layers $\times$ KV heads $\times$ head dimension), so long contexts and many concurrent users multiply quickly. Switching from multi-head to grouped-query attention reduces the number of KV heads and cuts the cache proportionally. Quantizing the cache, limiting the maximum context, or reducing batch size are further options. Note that quantizing only the *weights* would not fix this, since the weights were never the binding constraint.

Q63. Scenario: what would happen if you took a LLaMA model and replaced RoPE with learned absolute positional embeddings, then tried to serve $3\times$ the trained context length?

A. Positions beyond the trained maximum would have no learned embedding at all, so they would either be undefined or fall back to untrained vectors. Quality would degrade abruptly at the boundary rather than gracefully, and the model would effectively ignore or scramble the tail of the input. With RoPE the relative-offset formulation degrades more smoothly and can be extended by interpolating the rotation frequencies. This is precisely why relative and rotary schemes replaced absolute ones in long-context models.

8 Pretraining Objectives, Data, and Scaling Laws

Q64. State the pretraining objective of a decoder-only LLM precisely.

A. The objective is next-token prediction: given the prefix “The capital of Bangladesh is”, maximise the probability assigned to the true continuation “Dhaka”. Formally, minimise the cross-entropy between the model’s predicted distribution over the vocabulary and the one-hot target, summed over every position of every document. Training uses teacher forcing, feeding ground-truth tokens rather than the model’s own predictions. At inference the model generates autoregressively, feeding its own outputs back as input.

Q65. What issues arise from the standard pretraining data sources?

A. Common Crawl provides petabyte scale but very uneven quality; books, Wikipedia, arXiv/PubMed, and GitHub provide cleaner text but in far smaller quantities and with narrower coverage. The principal issues are diversity (over-representation of English and of certain domains and viewpoints), quality (spam, boilerplate, duplication, machine-generated text), and contamination (benchmark test sets appearing in the training corpus). Data quality matters more than raw quantity, so curation and deduplication are first-class design choices. Legal and privacy concerns around copyrighted or personal data are a further constraint.

Q66. What did Hoffmann et al. (2022) show, and what problem did it correct?

A. They showed that most existing models were compute-inefficient because they were too large for the amount of data they were trained on: model size and dataset size should be scaled roughly equally. Doubling the training data gives comparable or better gains than doubling the parameter count, so overparameterised models underperform smaller models trained on more tokens at equal compute. The demonstration was Chinchilla, a 70B model that outperformed

the 280B-parameter Gopher. This corrected the “bigger model is always better” assumption that drove the GPT-3 generation.

Q67. Scenario: you have a fixed compute budget and a team member proposes doubling model size while keeping the token count fixed. What would happen, and what should you do instead?

A. Doubling parameters at fixed data pushes the model further into the overparameterised regime, so it will underperform a smaller model trained on more tokens for the same compute, exactly the Chinchilla finding. Inference cost would also double permanently, which is worse than a one-off training cost. The compute-optimal move is to scale parameters and tokens together, or — if the deployment budget is what matters — to deliberately *undertrain* in parameters and *overtrain* in tokens, as LLaMA does, to get a small model that is cheap to serve. Whether the compute-optimal or the inference-optimal point is right depends on how many times the model will be run.

Q68. What can scaling laws *not* tell you?

A. They predict smooth trends in pretraining loss, not downstream capability on any specific task, and low perplexity does not guarantee reasoning, factuality, or safety. They assume a fixed data distribution and quality, so they say nothing about what happens when high-quality tokens run out or when the corpus contains model-generated text. They also ignore inference cost, alignment, and evaluation contamination. They are a budgeting tool, not a theory of capability.

9 Long Context and Mixture of Experts

Q69. What problem do long-context models solve, and what are their limitations?

A. The context window is the amount of text in tokens that a model can consider at once; extending it from 4K (GPT-3) to 128K (GPT-4 Turbo) to 1M+ (Gemini 1.5) makes whole-book analysis, full-codebase reasoning, and long legal contracts feasible without chunking. The limitations are that attention cost grows quadratically with length (partly mitigated by FlashAttention and related kernels), and that models exhibit the “lost in the middle” phenomenon, attending disproportionately to the beginning and end of the context. So a long window does not guarantee uniform use of it. The KV cache also grows linearly with context, dominating serving memory.

Q70. When should you *not* solve a problem by extending the context window?

A. When the knowledge base is much larger than any window, changes frequently, or must be auditable — retrieval is the right answer there. When per-query cost and latency matter, since stuffing 100K tokens into every request is expensive and slow even if it fits. When the relevant evidence is buried in the middle of a long input, because of the lost-in-the-middle effect. In short, a long window is for a genuinely long single document, not for a corpus.

Q71. What problem does Mixture of Experts solve, and how?

A. It decouples total parameter count from per-token compute, so a model can hold much more knowledge without proportionally more FLOPs per token. Many expert sub-networks replace the feed-forward block, and a router activates only a few per token; Mixtral 8x7B has 8 experts and activates 2, giving 47B total parameters but roughly 13B active. Only the feed-forward layers are expert-specific — token embeddings and attention layers are shared, which is why 8

experts of 7B do not sum to 56B. DeepSeek-V3 pushes this much further with 256 experts and 8 active.

Q72. What are the disadvantages and new problems of MoE?

A. All experts must be resident in memory even though only a few are used per token, so the memory footprint corresponds to the full 47B, not the 13B active. Routing must be load-balanced, or a few experts receive most tokens and the rest are wasted, which requires auxiliary balancing losses and adds training instability. Distributed serving becomes harder because expert parallelism introduces all-to-all communication. Fine-tuning is also more delicate, since routing decisions can shift and destabilise learned specialisation.

Q73. Scenario: you must maximise quality subject to a strict per-token latency budget, and memory is plentiful. Dense or MoE?

A. MoE is the better fit, because your binding constraint is compute per token and MoE buys extra capacity without extra active FLOPs. A dense model of equal quality would activate all its parameters on every token and miss the latency target. The price you pay is memory, which by assumption you have. If instead memory were the binding constraint — an edge device, say — the answer would invert and a smaller dense model, quantized and possibly distilled, would win.

10 Quantization, PEFT, Pruning, and Distillation

Q74. What problem does quantization solve, and what are its three concrete benefits?

A. It reduces the computational and memory cost of inference by representing weights and activations in low-precision types such as int8 instead of float32. Fewer bits mean the model needs less memory storage, consumes less energy in principle, and allows operations such as matrix multiplication to run much faster with integer arithmetic. This is what makes large models deployable on commodity or edge hardware. It targets inference cost, not training cost or model quality.

Q75. Compare post-training quantization and quantization-aware training.

A. Post-training quantization converts an already-trained model with no retraining, so it is fast and cheap but accuracy may drop, particularly at very low bit widths. Quantization-aware training simulates quantization during training — the forward pass uses quantized values while the backward pass keeps full precision — so the model learns weights that are robust to rounding. QAT generally recovers more accuracy but requires a training run and access to data. Use PTQ first and fall back to QAT only if the accuracy loss is unacceptable.

Q76. Write the affine int8 quantization scheme and explain why the zero-point is needed.

A. For a float x in a range $[a, b]$, the affine scheme is $x = S(x_q - Z)$ with quantized value $x_q = \text{round}(x/S + Z)$, where S is a positive float32 scale and Z is an int8 zero-point. int8 can represent only 256 values, so the problem is to project the float32 range $[a, b]$ onto that grid as faithfully as possible. The zero-point is the int8 code that maps exactly to float 0, which matters because zeros appear everywhere in ML — in padding, in ReLU outputs, in sparse weights — and a zero that quantizes to a nonzero value would inject systematic error. Without Z the scheme could only represent symmetric ranges around zero.

Q77. What is an accumulation data type and why is one needed?

A. It is the type used to hold the result of accumulating products and sums of the quantized values. For example, with two int8 values $A = 127$ and $B = 127$, the sum $C = A + B$ already exceeds the largest representable int8, so accumulating in int8 would overflow. Standard pairings are int8 with int32 accumulation, int16 with int32, float16 with float16, and bfloat16 with float32. Without a wider accumulator, precision loss would be severe enough to make the whole quantization exercise pointless.

Q78. Scenario: you convert a model from float32 to float16 and the loss becomes NaN inside the normalisation layers. Explain and fix.

A. LayerNorm uses a small epsilon, typically around 10^{-12} , but the smallest normally representable float16 value is around 6×10^{-5} , so epsilon underflows to zero and a division by a near-zero variance produces infinities and then NaNs. Very large activations can also overflow float16's limited range. The fix is mixed precision: keep the normalisation (and the accumulations) in float32 while storing and multiplying the bulk of the weights in float16, or use bfloat16, which has float32's exponent range at the cost of mantissa precision. The general lesson is that float32 $\rightarrow$ float16 is not automatically safe just because the two formats share a representation scheme.

Q79. What problem does quantization *not* solve?

A. It does not reduce training cost, since the model must be trained (or at least calibrated) first. It does not improve the model's knowledge, factuality, or reasoning — at best it preserves them. It does not help if the memory bottleneck is the KV cache rather than the weights, unless the cache itself is quantized. And it can silently degrade quality on the tail of the distribution, which is easy to miss if evaluation is limited to aggregate accuracy.

Q80. What problem does parameter-efficient fine-tuning solve, and how does LoRA work?

A. Full fine-tuning updates every parameter of a large model, which requires storing optimiser states for billions of parameters and produces a full-size copy of the model per task. PEFT freezes most of the pretrained parameters and layers and trains only a small set of task-specific parameters. Adapters do this by inserting small feed-forward layers between Transformer layers; LoRA does it by learning two low-rank matrices whose product is added to an existing weight matrix, so only the rank-decomposed factors are trained. The base weights are untouched, so one backbone can serve many tasks by swapping small adapter weights.

Q81. List the advantages of PEFT.

A. Increased efficiency and faster time-to-value, since far fewer parameters are updated and stored. No catastrophic forgetting, because the pretrained weights are frozen and the general capability is preserved. Lower risk of overfitting and lower data demands, since the small number of trainable parameters is a strong regulariser. And more accessible, more flexible AI: fine-tuning becomes possible on modest hardware, and many task-specific adapters can share one backbone.

Q82. Compare magnitude pruning and structured pruning, and state the standard pipeline.

A. The pipeline is: train the model, prune (for example remove 30% of the network), then fine-tune to recover accuracy. Magnitude pruning removes individual weights with the smallest absolute values, producing sparse weight matrices; structured pruning removes whole neurons,

attention heads, or layers. Magnitude pruning typically preserves accuracy better at a given sparsity, but unstructured sparsity yields little real speed-up without specialised hardware or kernels. Structured pruning is friendlier to hardware because the remaining computation is still dense, so it delivers actual latency gains.

Q83. Why can PEFT be insufficient for recovering accuracy after pruning?

A. Pruning physically removes capacity from the network, so the remaining weights must be reorganised, not merely nudged. PEFT constrains updates to a small, often low-rank, subspace, which may not contain the directions needed to compensate for deleted units. A full fine-tune has the freedom to redistribute the lost function across surviving parameters, but it is expensive. This is the practical tension the lecture highlights: the cheap recovery method may be too weak and the strong one too costly.

Q84. How does knowledge distillation work, and when is it the right tool?

A. A large teacher model produces soft labels — full probability distributions — over a dataset, and a small student is trained to match them by minimising the KL divergence between the two distributions. The soft targets carry more information than hard labels because they encode the teacher’s relative confidence across classes, which acts as a rich training signal. DistilBERT is about 40% smaller while retaining roughly 97% of BERT’s accuracy, and TinyLlama (1.1B) was trained from LLaMA 2 outputs. It is best for creating tiny, fast, specialised models such as on-device chat assistants.

Q85. Scenario: you must ship an assistant that runs entirely offline on a phone. Which combination of techniques would you use, and in what order?

A. Start with distillation to get a small student model from a large teacher, since architecture size is the dominant factor and no amount of quantization will shrink a 70B model to phone scale. Then apply structured pruning if further reduction is needed, with a short fine-tune to recover accuracy. Then quantize to int8 (or lower) for the final memory and energy reduction, checking numerically sensitive operations such as normalisation. Finally use PEFT adapters if the assistant must be personalised, since adapters are small enough to ship or update independently.

Q86. Scenario: 200 enterprise customers each want a model tuned to their own writing style. Full fine-tuning per customer is unaffordable. What do you do?

A. Train one shared frozen backbone and a LoRA adapter per customer, so storage per customer is megabytes rather than tens of gigabytes and adapters can be hot-swapped at serving time. Because the base weights never change, general capability is preserved and there is no catastrophic forgetting per customer. Training each adapter needs far less data, which suits customers with small style corpora. If the requirement were factual rather than stylistic, retrieval rather than PEFT would be the correct answer.

11 Instruction Tuning and Alignment

Q87. What problem does instruction tuning solve?

A. A pretrained language model is optimised to continue text, not to follow instructions, so prompted with “What is the sentiment of this review?” it may continue with more questions rather than answer. Instruction tuning fine-tunes the model with supervised learning on a labelled dataset of instructional prompts and corresponding outputs. This teaches it to respond

to task-specific natural-language prompts, making it general-purpose, user-friendly, and aligned with human intent. FLAN (2021) is the canonical early demonstration.

Q88. What is alignment, and what can supervised fine-tuning alone *not* achieve?

A. Alignment is the process of shaping model behaviour to be helpful, harmless, and honest according to human intentions and values. SFT is the easiest alignment method: collect human demonstrations of (prompt, ideal response), fine-tune with cross-entropy on the responses, and evaluate on held-out prompts. What SFT cannot do is teach *preferences*: it shows the model one acceptable answer but never says that answer A is better than answer B, so it cannot resolve trade-offs such as helpful versus harmless. It teaches format, not ranking.

Q89. Describe the RLHF pipeline and its practical difficulties.

A. Human annotators express pairwise preferences between candidate responses (A versus B); a reward model is trained to predict that preference score; then the policy is optimised against the reward model, classically with PPO. This produces the most helpful models in practice, as in ChatGPT and Claude. The difficulties are that preference collection is expensive, that PPO is notoriously unstable and hard to tune, and that the policy can exploit flaws in the reward model (reward hacking). Direct Preference Optimization removes the separate RL loop and is more practical for most teams.

Q90. What problem does Constitutional AI solve?

A. Standard RLHF requires humans to read and score thousands of toxic or dangerous outputs, which is expensive, slow, and harmful to the annotators. Constitutional AI, developed by Anthropic, instead gives the model a written constitution — a set of principles, rules, and guidelines — and has the model critique and revise its own outputs against it, then learns from that feedback via supervised and reinforcement learning. The loop is roughly constitution → red-teaming → self-critique → self-revision → learning from feedback. This dramatically reduces dependence on human-generated preference data while keeping the helpful, honest, harmless targets explicit and auditable.

Q91. Scenario: after instruction tuning, your assistant follows instructions well but is verbose and agrees with users even when they are wrong. Which stage should you add, and why?

A. Add a preference-learning stage: RLHF or DPO. The problem is not format — the model already answers in the right shape — but the relative ranking of behaviours, which SFT cannot express. Preference data lets you say that a concise, honest correction is better than a long, agreeable one. Note also that verbosity and sycophancy can be *introduced* by preference training if annotators reward length and agreement, so the reward signal itself must be designed against those biases.

12 Prompting and In-Context Learning

Q92. What is in-context learning, and what are its advantages?

A. In-context learning is the ability to learn a task from examples placed in the prompt, with no weight updates at all — one example (one-shot) or several (few-shot). Few-shot prompting often beats zero-shot by a significant margin. Its advantages are zero training cost and instant adaptability: a new task requires only a new prompt, so iteration takes seconds rather than a fine-tuning run. This is what made general-purpose LLM products possible.

Q93. What are the limitations of in-context learning?

A. Every example consumes context tokens and is re-processed on every request, so it costs more per query and competes with the actual input for window space. Performance is sensitive to example choice, order, and formatting, which makes it fragile and hard to reproduce. It does not durably teach the model anything: the “learning” vanishes when the prompt ends. For a high-volume, stable task, fine-tuning or PEFT is cheaper and more reliable than paying for demonstrations on every call.

Q94. What problem does Chain-of-Thought prompting solve, and why does it work?

A. CoT asks the model not only to answer but to generate the rationale leading to the answer, and it significantly improves performance on arithmetic, symbolic, and logical reasoning tasks. It works because a Transformer performs a fixed amount of computation per generated token, so a problem needing many sequential steps cannot be solved in one forward pass; emitting intermediate tokens effectively gives the model more serial computation and an external scratchpad. The intermediate steps also condition later steps, keeping the model on a coherent path. In short, it converts a hard one-step prediction into a sequence of easy ones.

Q95. What are the limitations of CoT prompting?

A. It only helps sufficiently large models; small models often produce fluent but wrong reasoning and can perform worse than with direct answering. The stated rationale is not guaranteed to be the model’s actual computation, so a plausible chain can accompany a wrong answer and give false confidence. It also multiplies output tokens, increasing latency and cost. And it does not help on tasks that are not decomposable into steps, such as simple factual recall.

Q96. Name the practical prompting techniques from the course and what each is for.

A. Role prompting (“You are a professional data scientist...”) sets register and domain assumptions; structured prompting (“Step 1: Analyze. Step 2: Propose. Step 3: Conclude.”) imposes a decomposition. Delimiters such as ### or triple quotes separate instructions from user input, which also reduces prompt-injection risk. Output formatting (“Respond with JSON”) makes responses machine-parseable; negative constraints suppress unwanted filler; and the escape hatch (“say I don’t know”) gives the model a licensed alternative to guessing, which reduces hallucination. Few-shot examples and CoT complete the toolkit.

13 Retrieval-Augmented Generation

Q97. What problems with pure LLMs does RAG address?

A. Pure LLMs hallucinate, have knowledge frozen at their training cutoff, have no access to private or enterprise data, and are expensive to retrain when facts change. RAG’s answer is not to retrain but to give the model a retriever. A retriever finds relevant documents from a knowledge base and a generator produces the answer conditioned on those retrieved documents. This keeps responses current and grounded without touching the model weights.

Q98. Describe the two components of a RAG system mechanically.

A. The retriever converts documents into vector embeddings and stores them in a vector database such as Pinecone, Weaviate, or FAISS; at query time it embeds the question and returns the top- k most similar documents. The generator is any LLM, which receives the retrieved documents plus the user query plus instructions, and is asked to answer conditioned

on that retrieved context. Because grounding does most of the work, small models around 7B often perform surprisingly well as generators when retrieval is good. The system’s quality is therefore bounded largely by retrieval quality.

Q99. What can RAG *not* solve?

A. It cannot fix reasoning: if the answer requires synthesising across many documents or performing multi-step inference, retrieval alone will not supply it. It cannot teach the model a style, format, or domain-specific behaviour — that is a fine-tuning problem. It fails when retrieval fails, and a confident answer built on the wrong top- k documents is arguably worse than no answer. It also adds latency, infrastructure, and a chunking/embedding pipeline that must itself be maintained.

Q100. Compare RAG and fine-tuning on knowledge freshness, transparency, compute cost, and hallucination risk.

A. Knowledge freshness: RAG updates by re-indexing documents, so it is current within minutes, while fine-tuning bakes knowledge into weights and needs a new training run. Transparency: RAG can cite the retrieved passage, whereas fine-tuned knowledge is opaque and unattributable. Compute cost: RAG has low one-off cost but higher per-query cost (embedding, search, longer prompts), while fine-tuning has high one-off cost and cheap inference. Hallucination risk: RAG lowers it by grounding the answer in retrieved text, but does not eliminate it, since the model may still ignore or misread the context.

Q101. Scenario (class exercise): you must build a model that imitates the writing style of a particular newspaper while also being factual. Which solution do you implement?

A. Both, because the requirement has two independent parts. Style is a behavioural pattern that must be internalised in the weights, so fine-tune (or use PEFT/LoRA) on that newspaper’s archive to acquire its voice, structure, and conventions. Factuality is about current, verifiable content, so use RAG at inference to ground each article in retrieved sources. Trying to solve style with retrieval would require stuffing examples into every prompt, and trying to solve factuality with fine-tuning would freeze the facts and invite hallucination.

Q102. When should you *not* use RAG?

A. When the required knowledge is small and static enough to fit in a system prompt, RAG’s infrastructure is pure overhead. When the task is generative or stylistic rather than factual — creative writing, code refactoring, tone rewriting — retrieval has nothing useful to retrieve. When latency budgets are extremely tight, the extra embedding and search round-trip may be unacceptable. And when the corpus is unstructured, low quality, or contradictory, retrieval will surface noise and can degrade an otherwise competent model.

14 Reasoning Models and Agentic AI

Q103. Why do standard LLMs fail at multi-step reasoning?

A. They behave like “System 1” thinkers: fast, automatic, pattern-matching, producing one answer in one forward pass with no ability to backtrack, revise, or plan. Because generation is autoregressive and committed, an early wrong token cannot be retracted and the rest of the answer is conditioned on the error. This is why they fail on multi-step maths, logic puzzles, planning, and self-correction. Chain-of-thought and reasoning models are attempts to add deliberate, “System 2” computation.

Q104. What is a reasoning model, and how does it differ from CoT prompting?

A. A reasoning model is an LLM fine-tuned to break complex problems into smaller steps — reasoning traces — before producing a final output. Unlike CoT prompting, which elicits step-by-step behaviour from a general model at inference time via the prompt, a reasoning model has been explicitly trained (often with reinforcement learning on verifiable outcomes) to produce and use such traces. The distinction is between pattern matching prompted into a step-by-step shape and deliberate reasoning trained into the weights. Reasoning models are a prerequisite for reliable agentic systems.

Q105. Describe the agent loop and why the whole industry converged on it.

A. At each iteration the agent assembles context from available inputs, invokes an LLM to reason and select an action, executes that action, observes the outcome, and feeds the observation back into the next iteration, repeating until the task is complete or a stopping condition is reached. This turns a single-shot predictor into a system that can gather information, act on the world, and correct itself. The convergence is explained by the pattern’s minimality: Agent = LLM + Memory + Planning + Tool Use covers essentially all useful autonomy. It also compensates directly for the model’s inability to backtrack within one forward pass.

Q106. What new problems does the agent loop give rise to?

A. Errors compound across iterations, so a wrong observation early can send the whole trajectory astray, and there is no guarantee of termination without an explicit stopping condition. Cost and latency scale with the number of iterations, and each iteration re-processes a growing context. Tool use introduces security exposure, including prompt injection through retrieved or fetched content. Evaluation also becomes much harder, since correctness is a property of a trajectory rather than of a single response.

15 Evaluation of LLMs

Q107. Why is evaluating generative models fundamentally harder than evaluating classifiers?

A. Generative models produce open-ended responses, so correctness is often subjective and there is no single reference answer to match against. Alignment is subjective too: how helpful a model is depends on who is asking and why. Benchmark contamination compounds the problem, because massively trained LLMs may already have seen the test data during pretraining, inflating scores without any real capability. Together these mean a single number rarely characterises a model.

Q108. Name the standard benchmarks and what each measures.

A. MMLU measures multitask accuracy across 57 subject areas; HumanEval measures code generation; BIG-Bench covers diverse reasoning tasks; GSM8K covers grade-school maths word problems; MT-Bench covers multi-turn conversation and reasoning. Together they span knowledge, coding, reasoning, and dialogue. The course’s caution is the important part: benchmarks are useful but not sufficient, since they are static, gameable, and vulnerable to contamination.

Q109. What are the advantages and disadvantages of LLM-as-a-judge?

A. Advantages: it is fast and scalable, aligns reasonably well with human preferences when prompted properly, is flexible across tasks, is consistent in the sense of applying the same criteria repeatedly, and can explain its verdicts. Disadvantages: verbosity bias (preferring

longer answers), egocentric or self-enhancement bias (preferring outputs resembling its own), ordering or position bias (favouring whichever candidate is presented first), non-trivial API cost, and general trustworthiness concerns about using a model to grade models. Mitigations include randomising presentation order, using rubrics, and calibrating against human labels. It is a scalable proxy, not a replacement for ground truth.

Q110. When is human evaluation necessary despite its cost?

A. It is the gold standard and for some qualities — humour, empathy, tone, cultural appropriateness — it may be the only valid measure, because no automatic metric captures them. It is also necessary for final safety and alignment judgements and for calibrating automatic judges. The drawbacks are that it is expensive, can be inconsistent and subjective across annotators, and scales poorly. The practical compromise is automatic metrics and LLM judges for iteration, with periodic human evaluation as the anchor.

16 Integrated Scenario Questions

Q111. Scenario: a hospital wants a system that answers clinician questions from its own constantly updated internal protocols, with citations. Which architecture, and what would go wrong with the alternatives?

A. RAG over the protocol corpus with a moderately sized instruction-tuned generator is the right choice, because it keeps answers current through re-indexing and can cite the retrieved passage, which the auditability requirement demands. Fine-tuning would freeze knowledge in the weights, would require retraining whenever a protocol changes, and could not attribute its answers. Simply extending the context window and pasting all protocols in would be expensive per query, would risk the lost-in-the-middle effect, and would not scale as the corpus grows. A small escape-hatch instruction (“answer only from the retrieved context; otherwise say you don’t know”) further reduces hallucination.

Q112. Scenario: your team fine-tunes a 7B model on 500 customer-service transcripts and it now answers in-domain well but has lost its general ability to follow unrelated instructions. What happened and how do you prevent it?

A. This is catastrophic forgetting: full fine-tuning updated every parameter on a narrow distribution, overwriting general instruction-following capability. With only 500 examples the model also overfits heavily to surface patterns in the transcripts. Using PEFT — LoRA or adapters — would freeze the pretrained weights and add a small trainable component, which the course lists explicitly as avoiding catastrophic forgetting and lowering data demands. Mixing in a small amount of general instruction data during training is a further mitigation.

Q113. Scenario: what would have happened if the Transformer had kept recurrence and simply added self-attention on top?

A. Long-range modelling would improve, since attention gives direct paths between distant tokens, but the sequential dependency would remain and training could still not be parallelised across positions. The compute-scaling curve that enabled GPT-scale pretraining would therefore not exist, and models would be limited to roughly the scale of RNN seq2seq systems. Practically, that architecture is the RNN-with-attention seq2seq model that preceded the Transformer. The historically decisive contribution was *removing* recurrence, not adding attention.

Q114. Scenario: a summarisation model produces fluent summaries containing facts absent from the source. Diagnose the cause and give two fixes at different levels.

A. The model is doing exactly what next-token prediction trains it to do: emit the most plausible continuation, which need not be the most faithful one. Exposure bias compounds this, since at inference the model conditions on its own possibly drifting output. One fix at the inference level is to constrain the prompt to answer only from the provided source, add an escape hatch, and lower the sampling temperature. One fix at the training level is preference optimisation (RLHF or DPO) with faithfulness as the preference criterion, since SFT alone cannot express “faithful beats fluent”.

Q115. Scenario: you must choose between a 13B dense model and a 47B MoE model with 13B active parameters for a service running on a single 40GB GPU. Which do you pick?

A. The 13B dense model, because the MoE requires all 47B parameters resident in memory even though only 13B are active per token, and that will not fit alongside the KV cache on a single 40GB card. The MoE’s advantage is quality per unit of compute, which only pays off when memory is not the binding constraint. If a second GPU were available and expert parallelism acceptable, the MoE would become attractive. Correctly identifying which resource is scarce — memory versus FLOPs — is the whole decision.

Q116. Scenario: your document classifier must handle inputs of 20,000 tokens but your BERT encoder accepts 512. Give three options with their trade-offs.

A. Chunk the document, encode each chunk, and pool or aggregate the chunk representations; this is cheap and preserves the pretrained model but loses cross-chunk dependencies. Use a long-context model with efficient attention; this preserves global context but costs more memory and compute and may exhibit lost-in-the-middle behaviour. Retrieve only the passages relevant to the classification decision and encode those; this is cheapest at inference but depends entirely on retrieval quality. The right choice depends on whether the label is determined locally (chunking suffices) or globally (long context is required).

Q117. Scenario: an evaluation shows your new model beating the previous one on MMLU by 8 points but users report it is worse. Explain the discrepancy and what you would measure instead.

A. MMLU is a static multiple-choice knowledge benchmark and may be contaminated, so a gain there can reflect memorisation rather than capability, and it says nothing about helpfulness, tone, instruction-following, or refusal behaviour. User-perceived quality is driven mostly by alignment properties that MMLU does not test. Measure with multi-turn and preference-based evaluation — MT-Bench-style multi-turn reasoning, pairwise LLM-as-a-judge comparisons with randomised ordering to control position bias, and a sample of human evaluation as the anchor. Task-specific offline metrics on real production prompts are more informative than any public leaderboard here.

Q118. Scenario: summarise, in one comparison, which architecture family you would pick for (a) named entity recognition, (b) machine translation, (c) an open-ended assistant, and why.

A. For named entity recognition use a bidirectional encoder such as BERT, because every token’s label benefits from both left and right context and the task is discriminative with abundant labelled data. For machine translation use an encoder–decoder, because there is a distinct source to be read bidirectionally and a distinct target to be generated, with cross-attention linking them — though a large decoder-only model with the source in the prompt is now competitive. For an open-ended assistant use a decoder-only model, because the task is generative, unbounded in scope, and benefits from in-context learning plus instruction tuning

and preference optimisation. The general principle is to match bidirectionality to understanding and causality to generation.